

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
Second Edition, O'Reilly Media 2020

Design Pattern Decorator and Factory (Simple Factory, Factory Method and Abstract Factory)

Francesco Guerra
francesco.guerra@unimore.it

The Decorator Pattern

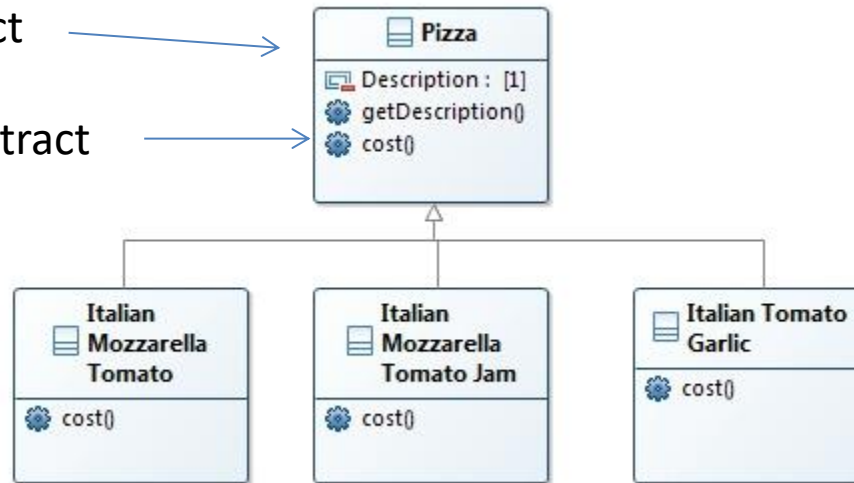
- ▶ With the techniques of decorating, you'll be able to give the objects new responsibilities without making any code changes to the underlying classes.
- ▶ Motivation
 - ▶ You have to model the “pizzas” sold by a company.
 - ▶ Bases (Italian, Deep Pan, Stuffed Crust, whole wheat, ...)
 - ▶ Toppings (mozzarella, pepperoni, mushroom, olive, pineapple).
 - ▶ Customers can combine bases and toppings as they prefer.
 - ▶ The final price of the pizza depends on the base and toppings selected.

Some combinations aren't allowed in real Italian Pizzas!
;-)

A possible representation

The Class is abstract

The method is abstract



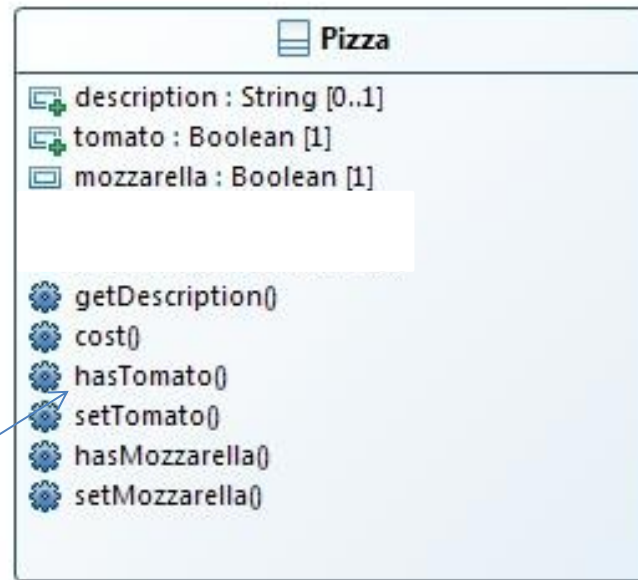
Each subclass implements **cost()** to return the cost of the pizza.

- ▶ What if we have a large amount of combinations?
- ▶ What happens when the price of mozzarella goes up?
- ▶ What if we add a new topping?

INGREDIENTI PASTA: farina tipo 0, tipo 00 o integrale (in base alla stagione), acqua, lievito, sale, olio extravergine di oliva. Tutte le pizze elencate hanno come ingredienti base pomodoro e mozzarella.					
PIZZA TRADIZIONALE					
1 ALTOPIANO	€ 6.00		19 GAMBERETTI	€ 5.50	
2 AMORE	€ 7.50		20 GORGONZOLA	€ 5.00	
3 ARTE PIZZA EXPRESS	€ 7.00		21 GUSTOSA	€ 6.00	
4 ASIAGO E PORCINI	€ 6.00		22 INVERNO	€ 6.00	
5 AUTUNNO	€ 6.00		23 LUCY	€ 7.00	
6 BRIE E PEPPERONI	€ 5.50		24 MARE	€ 7.00	
7 BRIE E SALAMINO	€ 5.50		25 MARGHERITA	€ 4.00	
8 CALABRESSE	€ 5.00		26 MARINARA	€ 3.00	
9 CALZONE	€ 6.00		27 MATTIA	€ 7.00	
10 CAMPANOLA	€ 6.00		28 MEDITERRANEA	€ 4.50	
11 CAPRESE	€ 6.00		29 MILANZANE	€ 4.50	
12 CAPRICCIOSA	€ 6.00		30 MELANZANE E GRANA	€ 5.50	
13 CHIARA	€ 6.50		31 MISTO FUNGHI	€ 6.50	
14 CRUDO	€ 5.50		32 MONTANA	€ 5.50	
15 DELIZIOSA	€ 6.50		33 MORENO	€ 7.00	
16 DIAVOLO	€ 4.50		34 NORDICA	€ 6.00	
17 ESTIVA	€ 6.00		35 PARMESANE	€ 6.50	
18 FUNGHI	€ 4.50		36 PATATE E WÜRSTEL	€ 5.00	
			37 PATATE E PORCHETTA	€ 5.50	
			38 PATATE E SALAMINO	€ 5.00	
			39 PEPPERONI	€ 4.50	
			40 POLPA DI GRANCHIO	€ 5.50	
			41 PORCHETTA	€ 5.00	
			42 PORCINI	€ 5.50	
			43 PROSCIUTTO	€ 4.50	
			44 PROSCIUTTO FUNGHI	€ 5.00	
			45 PUGLIESE	€ 4.50	
			46 QUATTRO FORMAGGI	€ 5.50	
			47 QUATTRO STAGIONI	€ 6.00	
			48 RICOTTA E SPINACI	€ 5.50	
			49 ROMANA	€ 4.50	
			50 RUCOLA E STRACCHINO	€ 5.50	
			51 SICILIANA	€ 5.50	
			52 SORRENTO	€ 6.50	
			53 TIROLESE	€ 6.00	
			54 TONNATA	€ 5.00	
			55 VALTELLINA	€ 7.00	
			56 VERDURE ALLA ORIGLIA	€ 6.00	
			57 VERDURE CRUDE	€ 6.00	
			58 VERDURE MISTE	€ 7.00	
			59 VIENNESE	€ 4.50	
			60 ZUCCHINE	€ 5.00	

Possible improvement?

- ▶ why do we need all these classes?
- ▶ Can't we just use instance variables and inheritance in the superclass to keep track of the condiments?



We have to create a subclass for each kind of base

The number of classes is reduced

Only a subset of the properties and methods has been introduced

Issue!

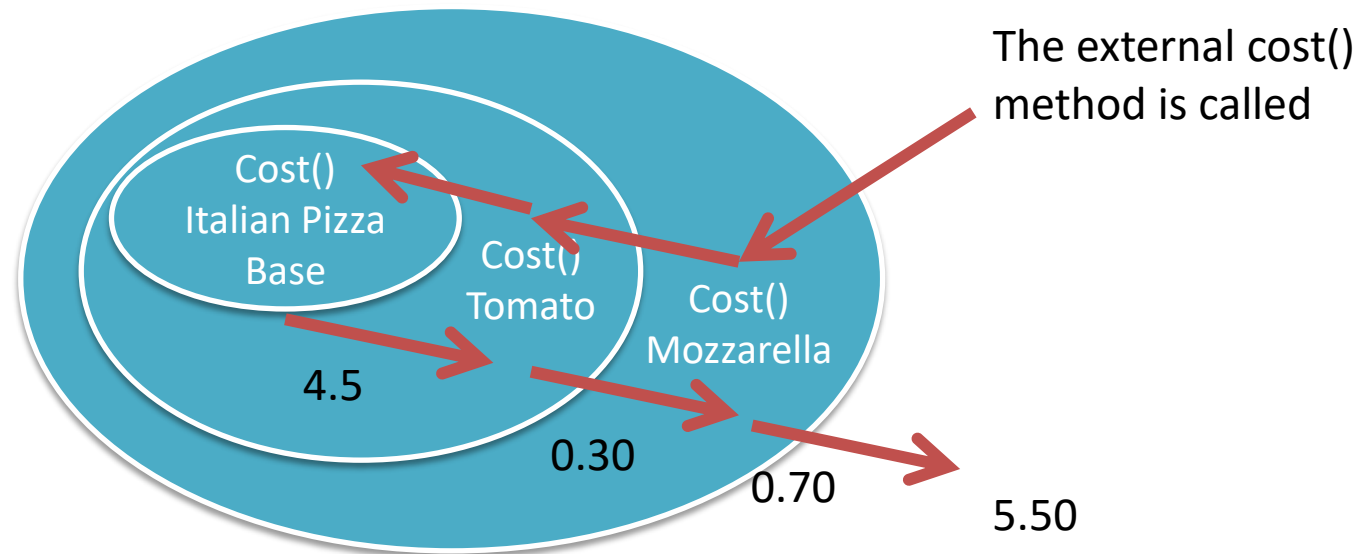
- ▶ Price changes for condiments will force us to alter existing code.
- ▶ New condiments will force us to add new methods and alter the cost method in the superclass.
- ▶ We may have new pizzas. For some of these pizzas (sweet pizzas - dessert pizzas), the condiments may not be appropriate.
 - ▶ Sweet pizzas subclass will still inherit methods like `hasMozzarella()`.
- ▶ What if a customer wants a double Mozzarella?

The Open-Closed Principle

- ▶ **Design Principle (Open-Closed Principle)**
Classes should be open for extension, but closed for modification.
 - ▶ While it may seem like a contradiction, there are techniques for allowing code to be extended without direct modification.
 - ▶ Applying the Open-Closed Principle EVERYWHERE is wasteful and unnecessary, and can lead to complex, hard-to-understand code.
 - ▶ There are some clever OO techniques for allowing systems to be extended, even if we can't change the underlying code.
 - ▶ In the Observer Pattern ... by adding new Observers, we can extend the Subject at any time, without adding code to the Subject.

The Decorator Pattern

- ▶ We start with a base and we “decorate it” run-time with condiments

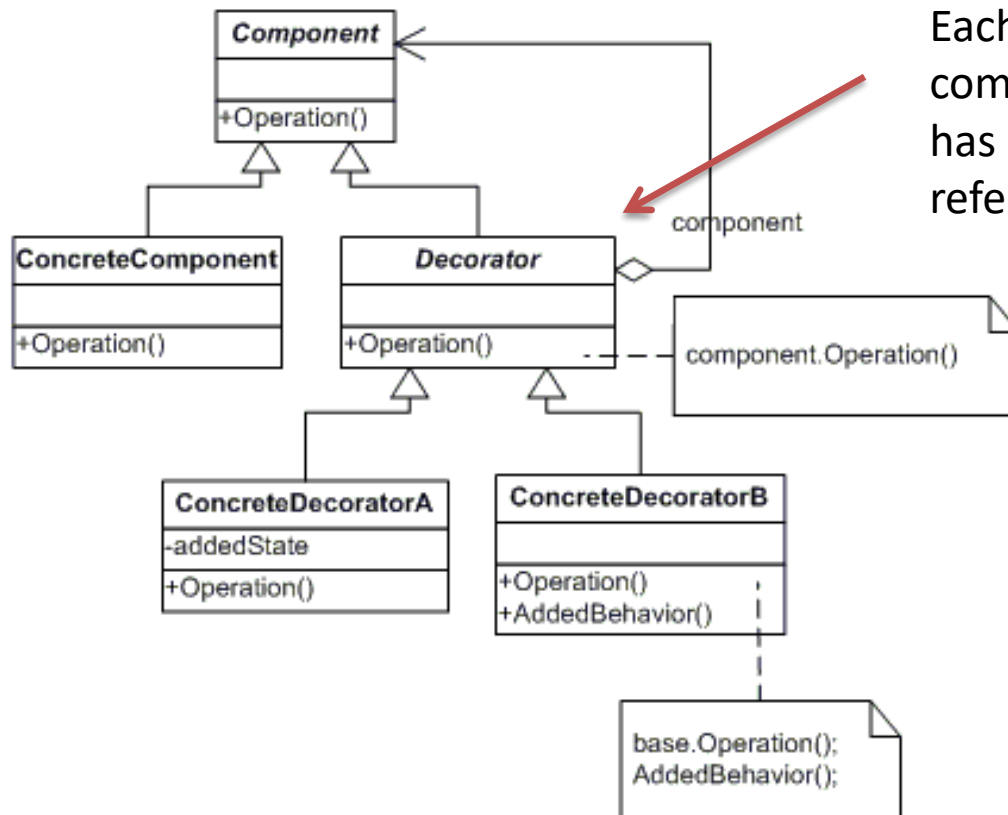


Key points

- ▶ Decorators have the same supertype as the objects they decorate.
- ▶ You can use one or more decorators to wrap an object.
- ▶ Given that the decorator has the same supertype as the object it decorates, we can pass around a decorated object in place of the original (wrapped) object.
- ▶ The decorator adds its own behavior either before and/or after delegating to the object it decorates to do the rest of the job.
- ▶ Objects can be decorated at any time, so we can decorate objects dynamically at runtime with as many decorators as we like.

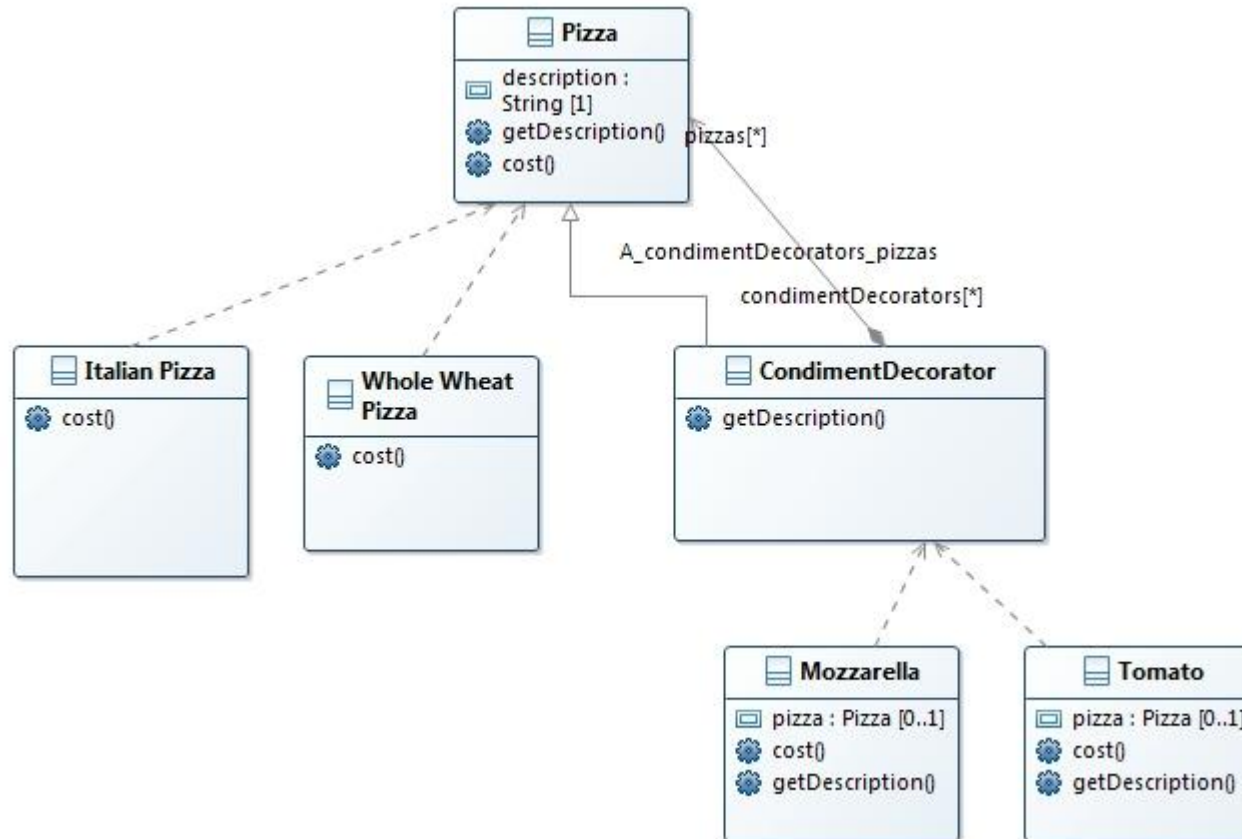
The Decorator Pattern

- ▶ The Decorator Pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.



Each decorator HAS-A (wraps) a component, which means the decorator has an instance variable that holds a reference to a component.

Decorating the Pizzas



The implementation

```
public abstract class Pizza{
    String description = "Unknown Pizza";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}
```

```
public abstract class CondimentDecorator extends Pizza{
    Pizza pizza;
    public abstract String getDescription();
}
```

Coding Pizzas and Condiments

```

public class Italian extends Pizza{
    public Italian() {
        description = "Italian base";
    }

    public double cost() {
        return 4.50;
    }
}

public class Tomato extends
CondimentDecorator {
    Pizza pizza;

    public Tomato(Pizza pizza) {
        this.pizza = pizza;
    }

    public String getDescription() {
        return pizza.getDescription() +
        ", Tomato";
    }

    public double cost() {
        return pizza.cost() + .20;
    }
}

```

► Francesco Guerra

```

public class WholeWheat extends Pizza{
    public WholeWheat () {
        description = " WholeWheat base";
    }

    public double cost() {
        return 5.30;
    }
}

public class Mozzarella extends
CondimentDecorator {
    Pizza pizza;

    public Mozzarella (Pizza pizza) {
        this.pizza = pizza;
    }

    public String getDescription() {
        return pizza.getDescription() +
        ", Mozzarella";
    }

    public double cost() {
        return pizza.cost() + .70;
    }
}

```

12

Serving Pizzas

```
public class PizzaRest{

    public static void main(String args[]) {
        Pizza pizza = new Italian();
        System.out.println(pizza.getDescription()
                           + " euro" + pizza.cost());

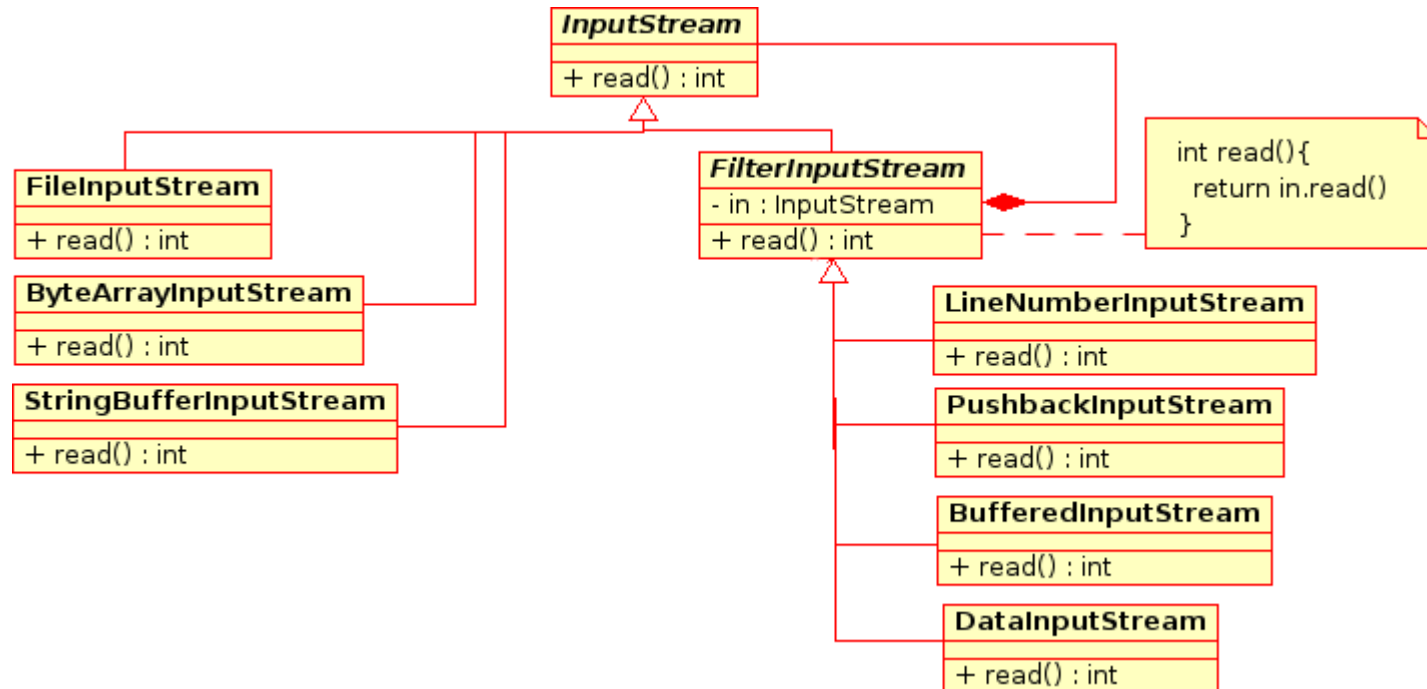
        Pizza pizza2 = new Italian();
        pizza2 = new Mozzarella(pizza2);
        pizza2 = new Mozzarella(pizza2);
        pizza2 = new Tomato(pizza2);
        System.out.println(pizza2.getDescription()
                           + " euro" + pizza2.cost());

    }

}
```

Real Example

► Java IO API



<http://zenit.senecac.on.ca/wiki/index.php/Decorator>

Key Points

- ▶ Inheritance is one form of extension, but not necessarily the best way to achieve flexibility in our designs.
- ▶ In our designs we should allow behavior to be extended without the need to modify existing code.
- ▶ Composition and delegation can often be used to add new behaviors at runtime.
- ▶ The Decorator Pattern provides an alternative to subclassing for extending behavior.
- ▶ The Decorator Pattern involves a set of decorator classes that are used to wrap concrete components.
- ▶ Decorator classes mirror the type of the components they decorate.
- ▶ Decorators change the behavior of their components by adding new functionality before and/or after (or even in place of) method calls to the component.
- ▶ You can wrap a component with any number of decorators.
- ▶ Decorators are typically transparent to the client of the component; that is, unless the client is relying on the component's concrete type.
- ▶ Decorators can result in many small objects in our design, and overuse can be complex.

Decorator Pattern Description - GoF structure (partial)

▶ Intent

- ▶ Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

▶ Also Known As

- ▶ Wrapper

▶ Applicability

▶ Use Decorator

- ▶ to add responsibilities to individual objects dynamically and transparently, that is, without affecting other objects.
- ▶ for responsibilities that can be withdrawn.
- ▶ when extension by subclassing is impractical. Sometimes a large number of independent extensions are possible and would produce an explosion of subclasses to support every combination. Or a class definition may be hidden or otherwise unavailable for subclassing

► Structure

► Participants

► Component

- defines the interface for objects that can have responsibilities added to them dynamically.

► ConcreteComponent

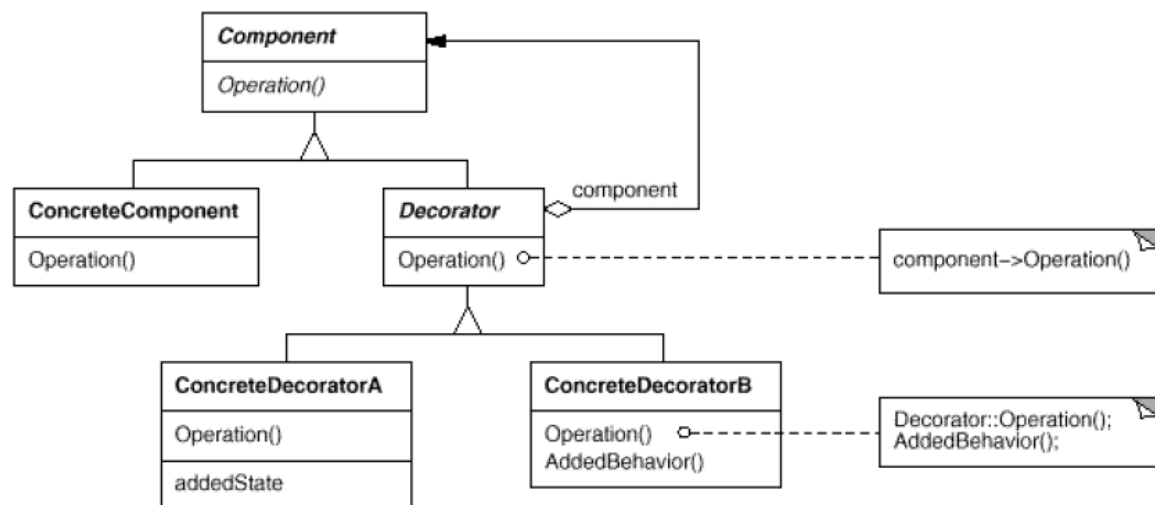
- defines an object to which additional responsibilities can be attached.

► Decorator

- maintains a reference to a Component object and defines an interface that conforms to Component's interface.

► ConcreteDecorator

- adds responsibilities to the component.



Decorator Pattern Description(3)

► Consequences

- *More flexibility than static inheritance.*
The Decorator pattern provides a more flexible way to add responsibilities to objects than can be had with static (multiple) inheritance.
 - With decorators, responsibilities can be added and removed at run-time simply by attaching and detaching them.
 - Decorators also make it easy to add a property twice.
- *Avoids feature-laden classes high up in the hierarchy.*
Decorator offers a pay-as-you-go approach to adding responsibilities.
 - Instead of trying to support all foreseeable features in a complex, customizable class, you can define a simple class and add functionality incrementally with Decorator objects.
- *A decorator and its component aren't identical.*
- *Lots of little objects.*
A design that uses Decorator often results in systems composed of lots of little objects that all look alike.
 - The objects differ only in the way they are interconnected, not in their class or in the value of their variables.

Design Pattern 4: Factory

- ▶ We aren't supposed to program to an implementation, but every time we use "new", that's exactly what we are doing.

```
Character character = new King();  
WheatherData station = new WheatherData();
```

- ▶ When you use new, you are definitely instantiating a concrete class, so that's an implementation, not an interface.

```
Character character  
...  
if (crown) {  
    character = new King();  
} else if (horse){  
    character = new Knight();  
} ...
```

When it comes time for changes or extensions, you'll have to reopen this code and examine what needs to be added (or deleted).

Remember that designs should be "open for extension but closed for modification"

Often this kind of code ends up in several parts of the application making maintenance and updates more difficult and error-prone.

What's wrong with “new”?

- ▶ Technically there's nothing wrong with new.
 - ▶ It's a fundamental part of Java.
 - ▶ The problem is to manage change and how change impacts our use of new
- ▶ If your code is written to an interface,
 - ▶ it will work with any new classes implementing that interface through polymorphism.
- ▶ When you have code that makes use of lots of concrete classes,
 - ▶ that code may have to be changed as new concrete classes are added.
 - ▶ The code will not be “closed for modification.” To extend it with new concrete types, you'll have to reopen it.
- ▶ So what can you do?
 - ▶ Design principle “identify the aspects that vary and separate them from what stays the same”.

Managing the concrete classes

- ▶ Let us suppose to have a pizza shop

```
Pizza orderPizza() {  
    Pizza pizza = new Pizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
  
    return pizza;  
}
```

Issue!

We will not become rich with only one kind of pizza!

Managing the concrete classes (2)

Type of pizza

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("greek")) {  
        pizza = new GreekPizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    }  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

Based on the type of pizza, the correct concrete class is instantiated

Each pizza here has to implement the Pizza interface.

This code is NOT closed for modification.
If the Pizza Shop changes its pizza offerings, we have to get into this code and modify it

We don't expect this code will change

Encapsulating object creation

- ▶ What we have to do is to take the creation code and move it out into another object that is only going to be concerned with

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("greek")) {  
        pizza = new GreekPizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    }  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

This will be part of an object Factory

Factories handle the details of object creation.
Once we have a SimplePizzaFactory, our orderPizza()
method just becomes a client of that object.

Building a PizzaFactory

```
public class SimplePizzaFactory {  
    public Pizza createPizza(String type) {  
        Pizza pizza = null;  
        if (type.equals("cheese")) {  
            pizza = new CheesePizza();  
        } else if (type.equals("peperoni")) {  
            pizza = new PeperoniPizza();  
        } else if (type.equals("clam")) {  
            pizza = new ClamPizza();  
        } else if (type.equals("veggie")) {  
            pizza = new VeggiePizza();  
        }  
        return pizza;  
    }  
}
```

Defining a simple factory as a static method is a common technique and is often called a static factory.

In this case you can't subclass and change the behavior of the create method (but you can call it without creating the factory object).

- ▶ By encapsulating the pizza creating in one class, we now have only one place to make modifications when the implementation changes.

The new PizzaStore Class with the Factory Object

```
public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

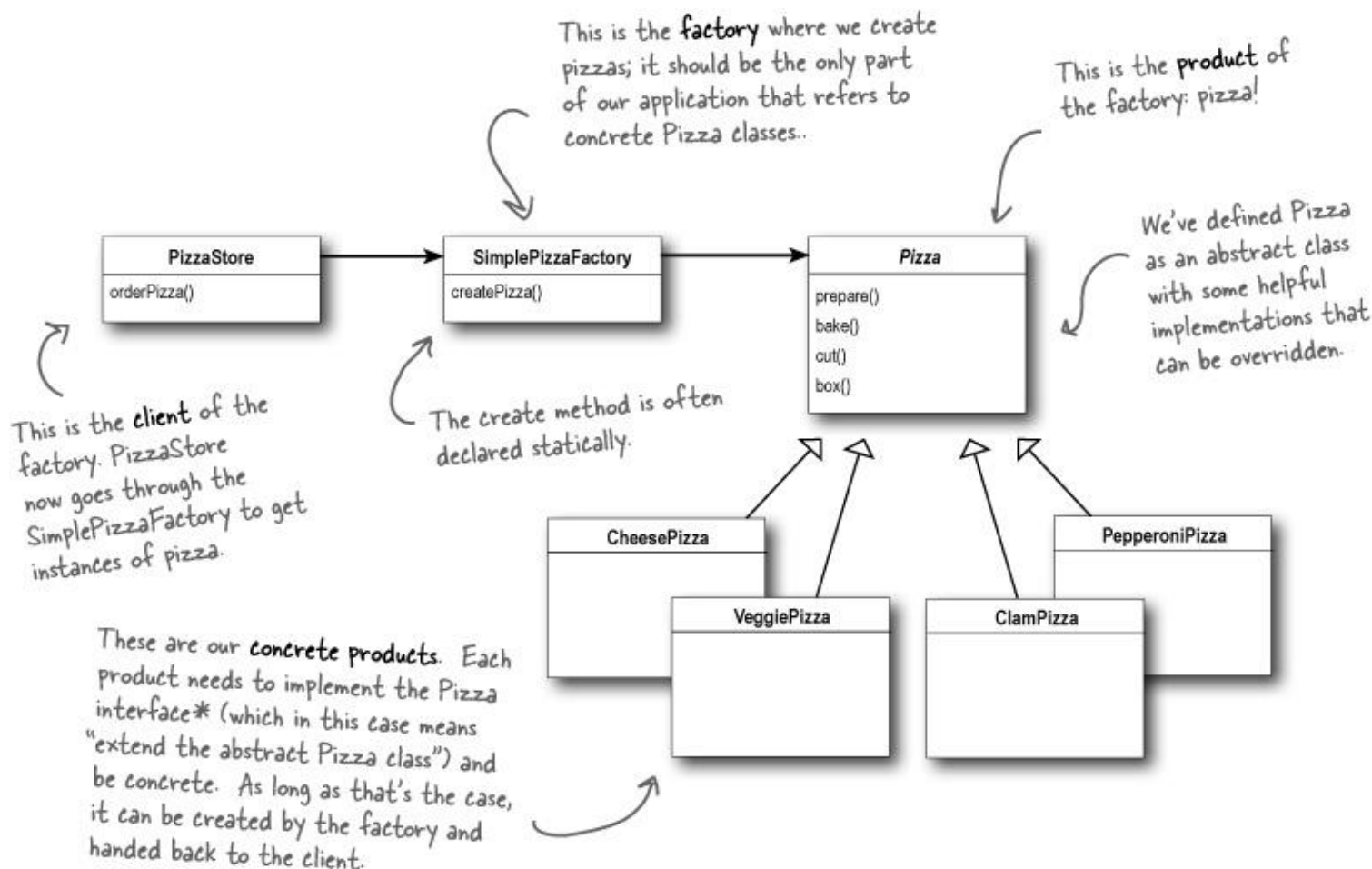
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
    // other methods here
}
```

The Simple Factory isn't actually a Design Pattern; it's more of a programming idiom.

Some developers do mistake this idiom for the "Factory Pattern," so be careful!



Simple Factory for the pizzas



Extending the Factory

- ▶ You want to extend the code for implementing it in different stores that implement different kinds of pizzas (in different regions, different kinds of pizzas are produced).
- ▶ Easy to manage!
 - ▶ We can create different factories, for managing each region / city

```
NaplesPizzaFactory naFactory = new NaplesPizzaFactory();  
PizzaStore naStore = new PizzaStore(naFactory);  
naStore.orderPizza("Veggie");
```

```
MilanPizzaFactory milanFactory = new MilanPizzaFactory();  
PizzaStore milanStore = new PizzaStore(milanFactory);  
milanStore.orderPizza("Veggie");
```

A new framework for the pizza store

We have to subclass the abstract class for each regional implementation

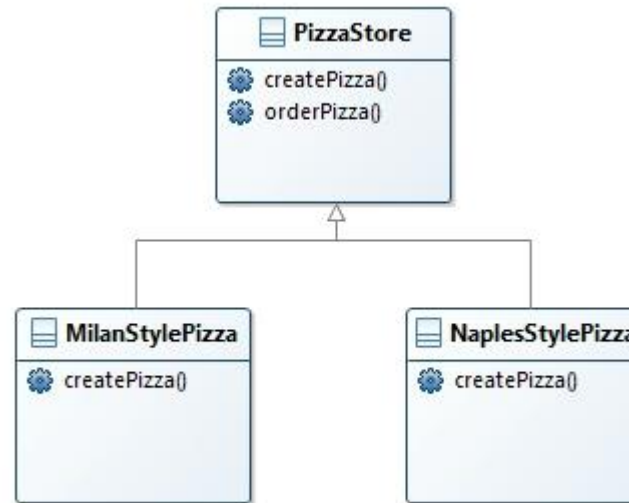
```
public abstract class PizzaStore {  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
        pizza = createPizza(type);  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
}
```

This is a call to a method, not a
call to a method in a Factory Object

```
abstract Pizza createPizza(String type);  
}
```

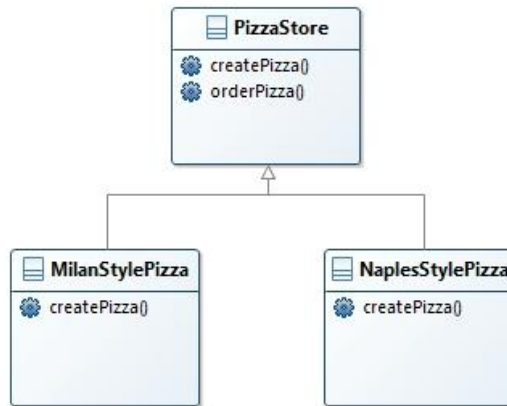
The Factory method is in the class

Allowing the subclasses to decide



- ▶ Each subclass provides an implementation of the `createPizza()` method, overriding the abstract `createPizza()` method in `PizzaStore`, while all subclasses make use of the `orderPizza()` method defined in `PizzaStore`.
- ▶ We could make the `orderPizza()` method final if we really wanted to enforce this.

Allowing the subclasses to decide (2)



- ▶ The orderPizza() method has no idea which subclass is actually running the code and making the pizzas.
 - ▶ In other words, it's decoupled!
- ▶ When orderPizza() calls createPizza(), one of your subclasses will be called into action.
- ▶ The kind of pizza is decided by the choice of pizza store you order from, MilanStylePizza or NaplesStylePizza.

A “real” PizzaStore

```
public class NaplesStylePizzaStore extends PizzaStore {  
    Pizza createPizza(String item) {  
        if (item.equals("cheese")) {  
            return new NaplesStyleCheesePizza();  
        } else if (item.equals("veggie")) {  
            return new NaplesStyleVeggiePizza();  
        } else if (item.equals("clam")) {  
            return new NaplesStyleClamPizza();  
        } else if (item.equals("pepperoni")) {  
            return new NaplesStylePepperoniPizza();  
        } else return null;  
    }  
}
```

For each type of Pizza we create the Naples style.

- ▶ Note that the `orderPizza()` method in the superclass has no clue which Pizza we are creating;
 - ▶ it just knows it can prepare, bake, cut, and box it!

The abstract Class PizzaStore

```
public abstract class PizzaStore {  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
        pizza = createPizza(type);  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
    protected abstract Pizza createPizza(String type);  
  
    // other methods here  
}
```

- ▶ A factory method handles object creation and encapsulates it in a subclass. This decouples the client code in the superclass from the object creation code in the subclass.

Details

A factory method returns a Product that is typically used within methods defined in the superclass.

A factory method may be parameterized (or not) to select among several variations of a product.

`abstract Product factoryMethod(String type)`

A factory method is abstract so the subclasses are counted on to handle object creation.

A factory method isolates the client from knowing what kind of concrete Product is actually created.

Ordering pizzas with the pizza factory method

- ▶ Create an instance of the desired PizzaStore.

```
PizzaStore naplesPizzaStore = new NaplesPizzaStore();
```

- ▶ You can call the orderPizza() method and pass in the type of pizza you want

```
naplesPizzaStore.orderPizza("cheese");
```

- ▶ There is a specific method for each subclass of PizzaStore. The Pizza is returned to the orderPizza() method.

```
Pizza pizza = createPizza("cheese");
```

- ▶ The orderPizza() method has no idea what kind of pizza was created, but it knows it is a pizza and it prepares, bakes, cuts, and boxes it.

```
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();
```

The Pizza Abstract Class

```
public abstract class Pizza {
    String name;
    String dough;
    String sauce;
    ArrayList<String> toppings = new ArrayList<String>();

    void prepare() {
        System.out.println("Preparing " + name);
        System.out.println("Adding toppings: ");
        for (String topping : toppings) {
            System.out.println("    " + topping);
        }
    }
    void bake() {System.out.println("Bake for 25 minutes at 350");}
    void cut() {System.out.println("Cutting the pizza into slices");    }
    void box() {System.out.println("Place pizza in official PizzaStore
                                box");
    }
    public String getName() {
        return name;
    }
}
```

Defining Pizzas

```
public class NaplesStyleCheesePizza extends Pizza {
    public NaplesStyleCheesePizza() {
        name = "Naples Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara Sauce";
        toppings.add("Mozzarella Cheese");
    }
}

public class MilanStyleCheesePizza extends Pizza {
    public MilanStyleCheesePizza() {
        name = "Milan Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";

        toppings.add("Grated Cheese");
    }
    void cut() {
        System.out.println("Cutting the pizza into squared slices");
    }
}
```

Testing it

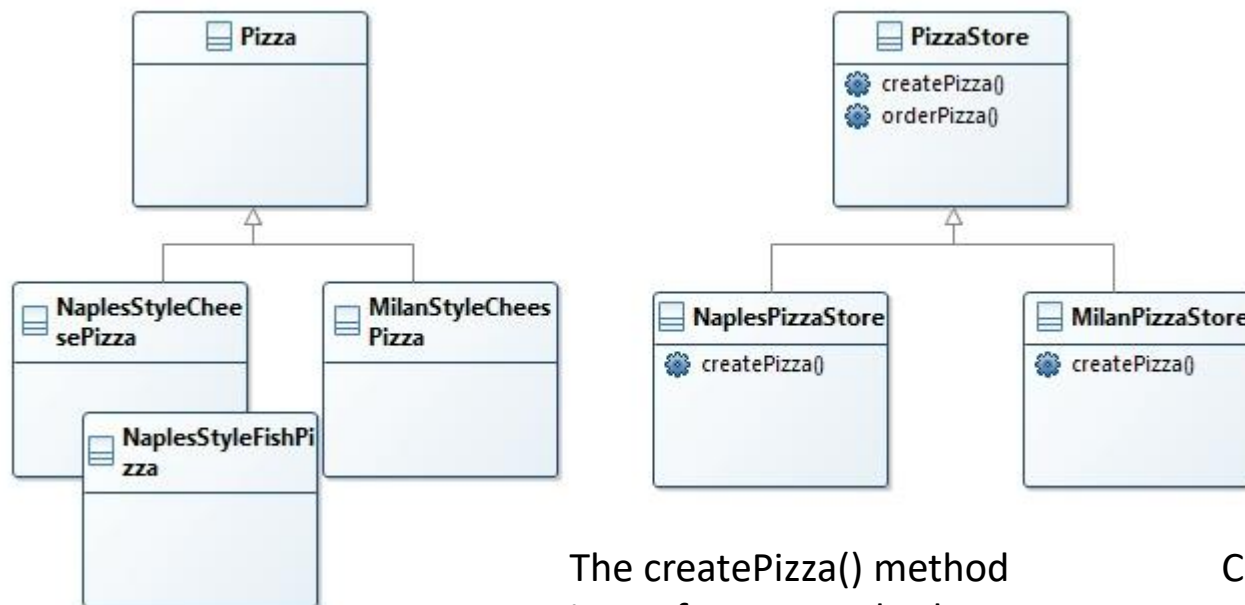
```
public class PizzaTest {  
  
    public static void main(String[] args) {  
        PizzaStore naStore = new NaplesPizzaStore();  
  
        Pizza pizza = naStore.orderPizza("cheese");  
        System.out.println("You ordered a " + pizza.getName() +  
                            "\n");  
    }  
}
```

The representation

Factories produce products, and in the PizzaStore, our product is a Pizza.

This is our abstract creator class. It defines an abstract factory method that the subclasses implement to produce products.

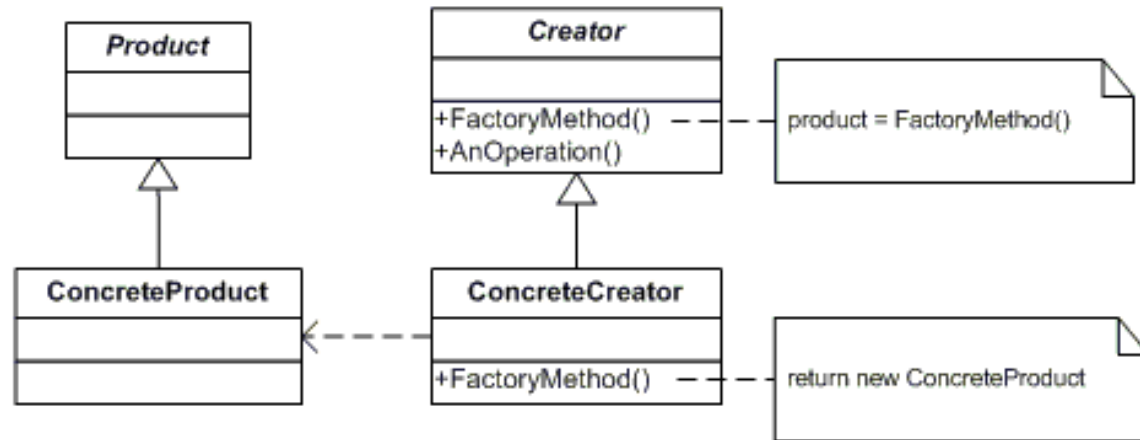
Often the creator contains code that depends on an abstract product, which is produced by a subclass. The creator never really knows which concrete product was produced



The **createPizza()** method is our factory method. It produces products.

Classes that produce products are called concrete creators.

The Factory Method Pattern



- ▶ The Factory Method Pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate.
 - ▶ Factory Method lets a class defer instantiation to subclasses.
 - ▶ You say “decide” not because the pattern allows subclasses themselves to decide at runtime, but because the creator class is written without knowledge of the actual products that will be created, which is decided purely by the choice of the subclass that is used.

Object dependencies

- ▶ If you don't use a Factory, you instantiate an object, dependent on its concrete class.
- ▶ Any changes to the concrete implementations of characters affects the CharacterStore,
 - ▶ The CharacterStore “depends on” the character implementations.
- ▶ If the implementation of the concrete classes changes, then we may have to modify in CharacterStore.
- ▶ Every new kind of character we add creates another dependency for CharacterStore.

```
Character character
...
if (crown) {
    character = new King()
} else if (horse){
    character = new Knight()
} ...
```

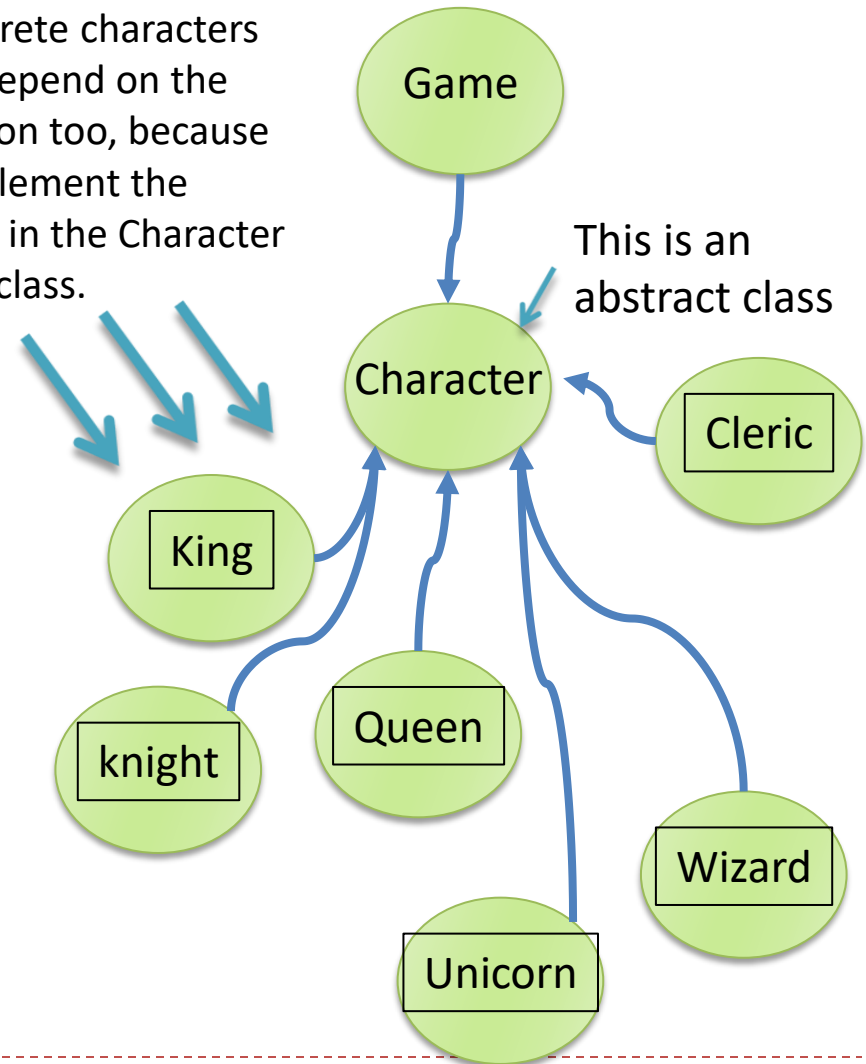
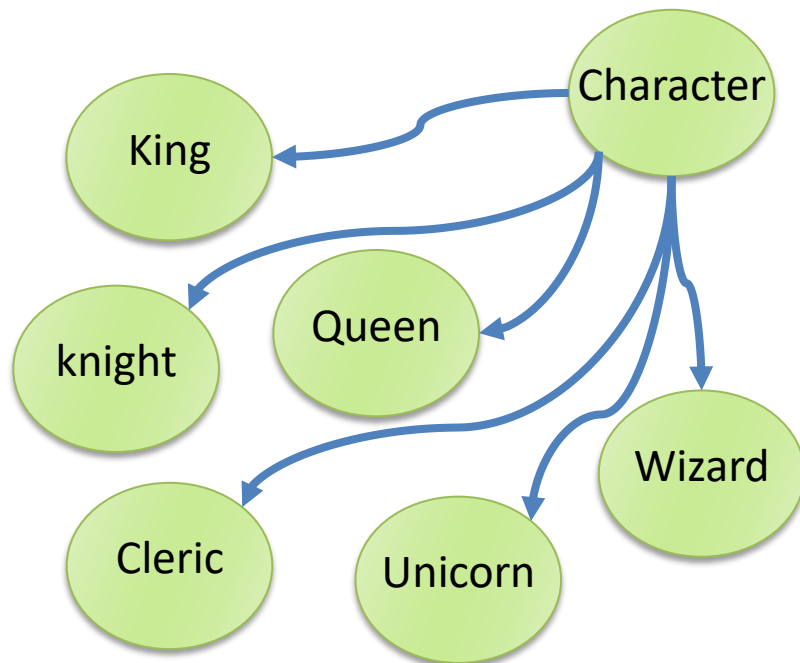

The Dependency Inversion Principle

- ▶ **Dependency Inversion Principle.**
Depend upon abstractions. Do not depend upon concrete classes
 - ▶ It suggests that our high-level components should not depend on our low-level components; rather, they should both depend on abstractions.
- ▶ Now, this principle tells us we should instead write our code so that we are depending on abstractions, not concrete classes.
 - ▶ That goes for both our high-level modules and our low-level modules.

Applying the Principle

```
Character character
...
if (crown) {
  character = new King()
} else if (horse) {
  character = new Knight()
} ...
```

The concrete characters classes depend on the abstraction too, because they implement the interface in the Character abstract class.



```

public class DependentPizzaStore {

    public Pizza createPizza(String style, String type) {
        Pizza pizza = null;
        if (style.equals("NY")) {
            if (type.equals("cheese")) {
                pizza = new NYStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new NYStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new NYStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new NYStylePepperoniPizza();
            }
        } else if (style.equals("Chicago")) {
            if (type.equals("cheese")) {
                pizza = new ChicagoStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new ChicagoStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new ChicagoStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new ChicagoStylePepperoniPizza();
            }
        } else {
            System.out.println("Error: invalid type of pizza");
            return null;
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}

```

You can write your answers here:

number

n

In the pizza example

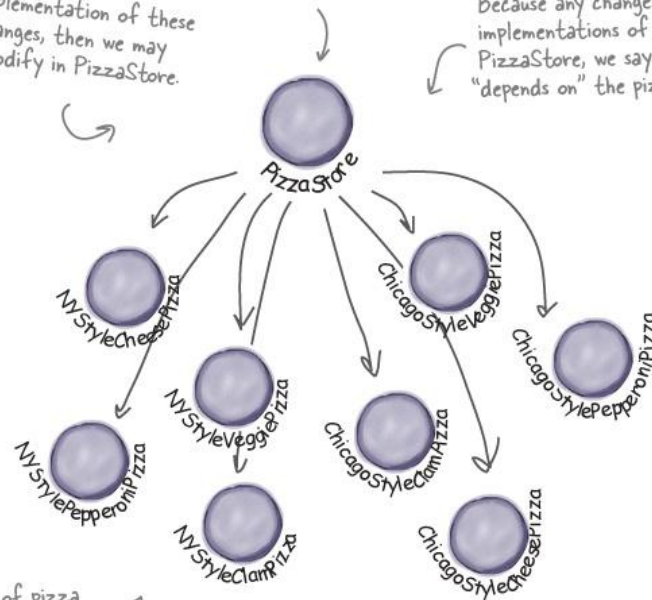
Handles all the NY style pizzas

Handles all the Chicago style pizzas

This version of the PizzaStore depends on all those pizza objects, because it's creating them directly.

If the implementation of these classes changes, then we may have to modify in PizzaStore.

Because any changes to the concrete implementations of pizzas affects the PizzaStore, we say that the PizzaStore "depends on" the pizza implementations.



Every new kind of pizza we add creates another dependency for PizzaStore.



Francesco Guerra



The “inversion” in Dependency Inversion Principle

- ▶ The low-level components now depend on a higher level abstraction.
- ▶ Likewise, the high-level component is also tied to the same abstraction.
- ▶ How to avoid OO designs that violate the Dependency Inversion Principle
 - ▶ No variable should hold a reference to a concrete class.
 - ▶ If you use new, you’ll be holding a reference to a concrete class. Use a factory to get around that!
 - ▶ No class should derive from a concrete class.
 - ▶ If you derive from a concrete class, you’re depending on a concrete class. Derive from an abstraction, like an interface or an abstract class.
 - ▶ No method should override an implemented method of any of its base classes.
 - If yes, then your base class wasn’t really an abstraction to start with. Those methods implemented in the base class are meant to be shared by all your subclasses.
- ▶ This is a guideline you should strive for, rather than a rule you should follow all the time.
 - ▶ Clearly, every single Java program ever written violates these guidelines

- ▶ No method should override an implemented method of any of its base classes.
 - ▶ This is possible!

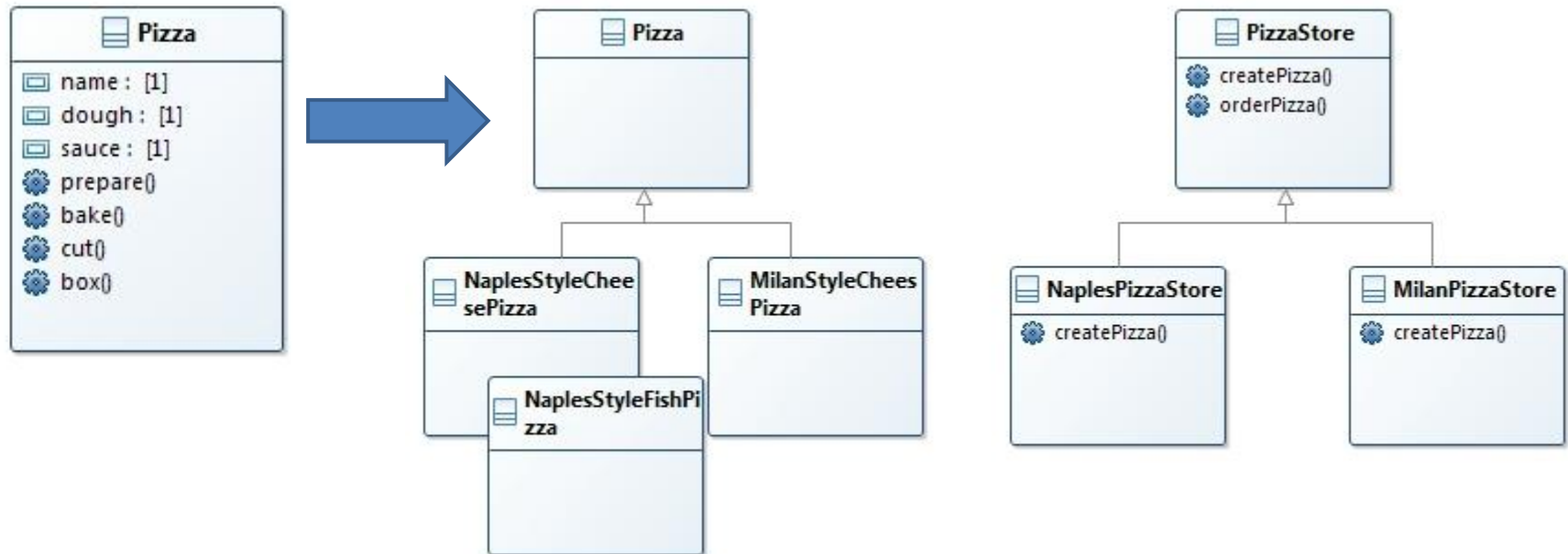
```
public class A{  
    public void a(){  
    }  
}  
  
public class B extends A{  
    @Override  
    public void a(){  
    }  
}
```

► Use a Strategy Pattern!

```
interface DescriptionProvider {  
    String getDescription(Element e);  
}  
  
class A {  
    private DescriptionProvider provider=new DefaultDescriptionProvider();  
  
    public final String getDescription(Element e) {  
        return provider.getDescription(e);  
    }  
  
    public final void setDescriptionProvider(@NotNull DescriptionProvider provider) {  
        this.provider=provider;  
    }  
}  
  
class B extends A {  
    public B() {  
        setDescriptionProvider(new CustomDescriptionProvider());  
    }  
}
```

<https://softwareengineering.stackexchange.com/questions/315344/is-overriding-concrete-methods-a-code-smell>

The PizzaStore Again



- ▶ We should ensuring consistency in the ingredients
 - ▶ Each family (Store) consists of a type of dough, sauce, cheese
- ▶ We could build a factory to create the ingredients

The ingredient factories

```
public interface PizzaIngredientFactory {  
  
    public Dough createDough();  
    public Sauce createSauce();  
    public Cheese createCheese();  
    public Veggies[] createVeggies();  
    public Pepperoni createPepperoni();  
    public Clams createClam();  
  
}
```

← Lots of new classes here, one per ingredient

- ▶ Build a Factory for each region.
 - ▶ You'll create a subclass of `PizzaIngredientFactory` that implements each create method.
- ▶ Implement a set of ingredient classes to be used with the Factory, like `ReggianoCheese`. These classes can be shared among regions where appropriate.
- ▶ Then we still need to hook all this up by working our new ingredient Factories into the old `PizzaStore` code.

Building the Naples ingredient factory

```
public class NaplesPizzaIngredientFactory implements PizzaIngredientFactory {

    public Dough createDough() {
        return new ThinCrustDough();
    }
    public Sauce createSauce() {
        return new MarinaraSauce();
    }
    public Cheese createCheese() {
        return new MozzarellaCheese();
    }
    public Veggies[] createVeggies() {
        Veggies veggies[] = { new Garlic(), new Onion(), new Mushroom(), new
RedPepper() };
        return veggies;
    }
    public Pepperoni createPepperoni() {
        return new SlicedPepperoni();
    }
    public Clams createClam() {
        return new FreshClams();
    }
}
```



The new Pizza Abstract Class

```
public abstract class Pizza {  
    String name;  
    Dough dough;  
    Sauce sauce;  
    Veggies veggies[];  
    Cheese cheese;  
    Pepperoni pepperoni;  
    Clams clam;  
    abstract void prepare();  
    void bake() {System.out.println("Bake for 25 minutes at 350");    }  
    void cut() {  
        System.out.println("Cutting the pizza into diagonal slices");  
    }  
    void box() {  
        System.out.println("Place pizza in official PizzaStore box");  
    }  
    void setName(String name) {this.name = name;    }  
    String getName() {return name;    }  
    public String toString() {  
        // code to print pizza here  
    }  
}
```

We've now made the prepare method abstract. This is where we are going to collect the ingredients needed for the pizza, which of course will come from the ingredient factory.

The concrete Pizzas

```
public class CheesePizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;

    public CheesePizza(PizzaIngredientFactory ingredientFactory) {
        this.ingredientFactory = ingredientFactory;
    }

    void prepare() {
        System.out.println("Preparing " + name);
        dough = ingredientFactory.createDough();
        sauce = ingredientFactory.createSauce();
        cheese = ingredientFactory.createCheese();
    }
}
```

- ▶ When we wrote the Factory Method code, we had a NaplesCheesePizza and a MilanCheesePizza class.
 - ▶ The only thing that differs is the use of regional ingredients.

Revisiting the pizza stores

```
public class NaplesPizzaStore extends PizzaStore {

    protected Pizza createPizza(String item) {
        Pizza pizza = null;
        PizzaIngredientFactory ingredientFactory =
            new NaplesPizzaIngredientFactory();

        if (item.equals("cheese")) {

            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("Naples Style Cheese Pizza");

        } else if (item.equals("clam")) {

            pizza = new ClamPizza(ingredientFactory);
            pizza.setName(" Naples Style Clam Pizza");

        } else if (item.equals("...")) {
            ...
        }
        return pizza;
    }
}
```

What we have done

- ▶ We provided a means of creating a family of ingredients for pizzas by introducing a new type of factory called an Abstract Factory.
- ▶ An Abstract Factory gives us an interface for creating a family of products.
 - ▶ By writing code that uses this interface, we decouple our code from the actual factory that creates the products.
 - ▶ That allows us to implement a variety of factories that produce products meant for different contexts—such as different regions, different operating systems, or different look and feels.
- ▶ Because the code is decoupled from the actual products, we can substitute different factories to get different behaviors (like getting marinara instead of plum tomatoes).

How to serve pizzas

- ▶ First we need a Naples PizzaStore:

```
PizzaStore naplesPizzaStore = new NaplesPizzaStore();
```

- ▶ Now that we have a store, we can take an order:

```
naplesPizzaStore.orderPizza("cheese");
```

- ▶ The orderPizza() method first calls the createPizza() method:

```
Pizza pizza = createPizza("cheese");
```

- ▶ When the createPizza() method is called, that's when our ingredient factory gets involved:

```
Pizza pizza = new CheesePizza(naplesIngredientFactory);
```

The ingredient factory is chosen and instantiated in the PizzaStore and then passed into the constructor of each pizza

How to serve pizzas (2)

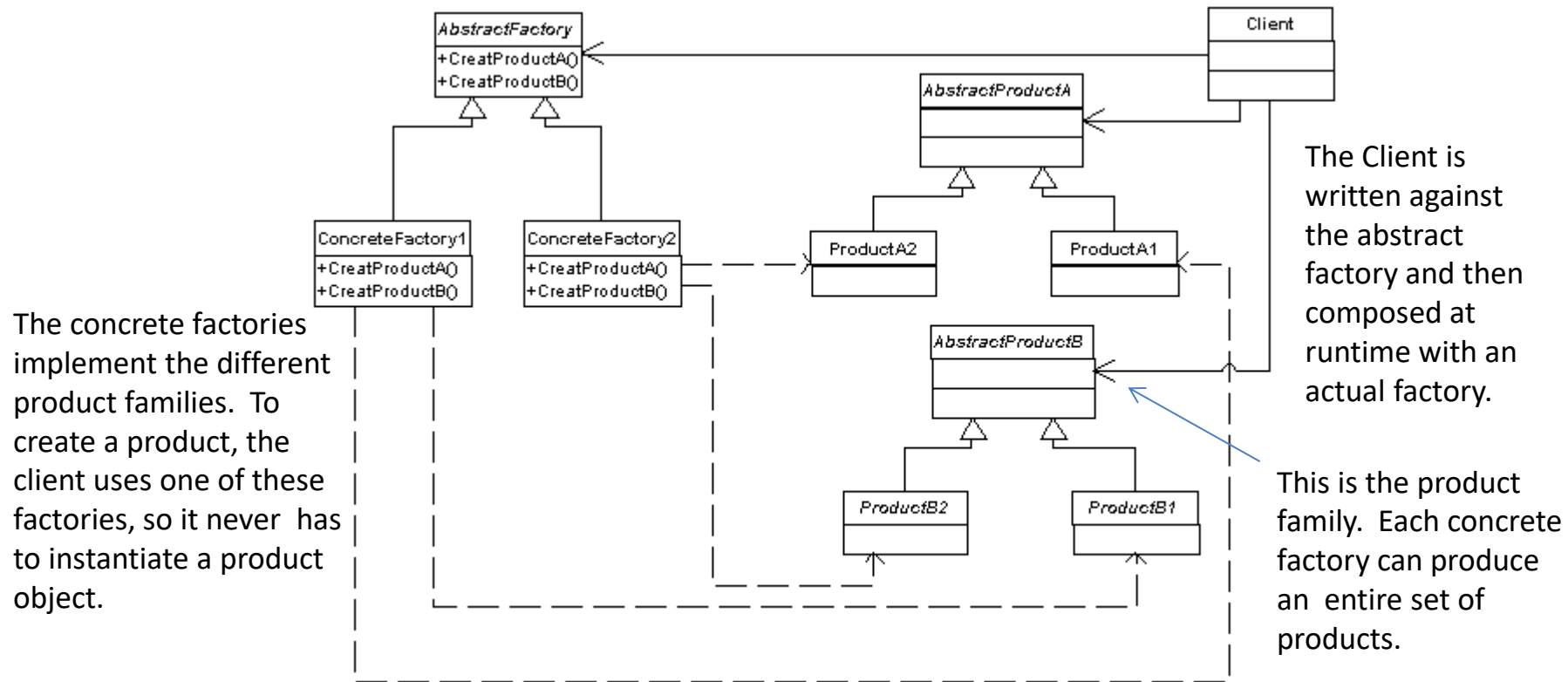
- ▶ Next we need to prepare the pizza. Once the prepare() method is called, the factory is asked to prepare ingredients:

```
void prepare() {  
    dough = factory.createDough();  
    sauce = factory.createSauce();  
    cheese = factory.createCheese();  
}
```

- ▶ Finally, we have the prepared pizza in hand and the orderPizza() method bakes, cuts, and boxes the pizza.

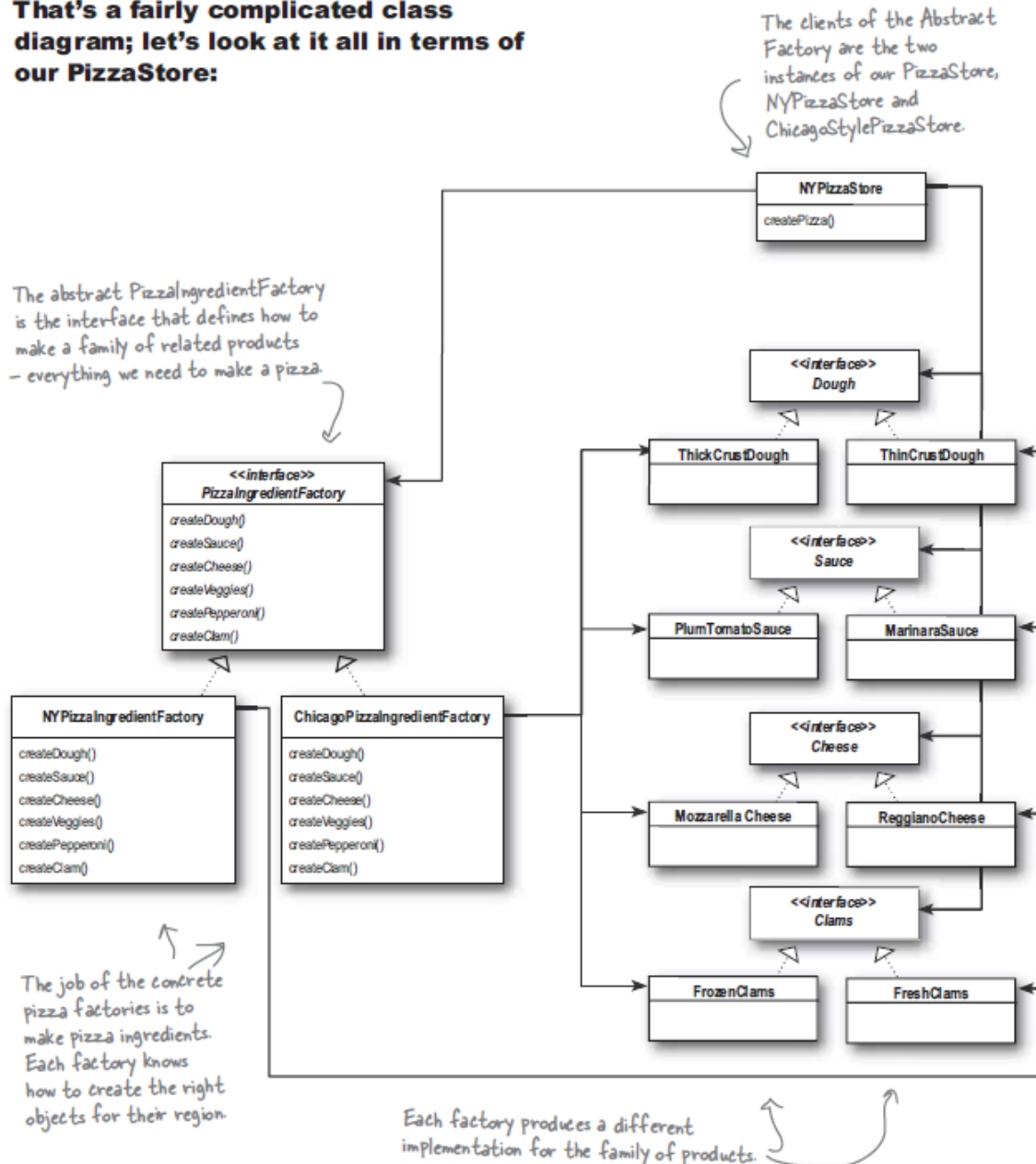
Abstract Factory Pattern defined

- ▶ The Abstract Factory Pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.



That's a fairly complicated class diagram; let's look at it all in terms of our **PizzaStore**:

The abstract **PizzaIngredientFactory** is the interface that defines how to make a family of related products - everything we need to make a pizza.



Factory Method vs Abstract Factory

- ▶ One **difference** between the two is that with the **Abstract Factory pattern**, a class delegates the responsibility of object instantiation to another object via composition whereas the **Factory Method pattern** uses inheritance and relies on a subclass to handle the desired object instantiation. [stackoverflow]

```
public class NaplesStylePizzaStore extends PizzaStore {
    Pizza createPizza(String item) {
        if (item.equals("cheese")) {
            return new NaplesStyleCheesePizza();
        } else if (item.equals("veggie")) {
            return new NaplesStyleVeggiePizza();
        } else if (item.equals("clam")) {
            return new NaplesStyleClamPizza();
        } else if (item.equals("pepperoni")) {
            return new NaplesStylePepperoniPizza();
        } else return null;
    }
}
```

Factory Method

Abstract Factory

```
public class NaplesPizzaStore extends PizzaStore {

    protected Pizza createPizza(String item) {
        Pizza pizza = null;
        PizzaIngredientFactory ingredientFactory =
            new NaplesPizzaIngredientFactory();

        if (item.equals("cheese")) {

            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("Naples Style Cheese Pizza");

        } else if (item.equals("clam")) {

            pizza = new ClamPizza(ingredientFactory);
            pizza.setName("Naples Style Clam Pizza");

        } else if (item.equals("...")) {
            ...
        }
        return pizza;
    }
}
```



Key Points

- ▶ All factories encapsulate object creation.
 - ▶ Simple Factory is a simple way to decouple your clients from concrete classes.
 - ▶ Factory Method relies on inheritance: object creation is delegated to subclasses, which implement the factory method to create objects.
 - ▶ Abstract Factory relies on object composition: object creation is implemented in methods exposed in the factory interface.
- ▶ All factory patterns promote loose coupling by reducing the dependency of your application on concrete classes.
 - ▶ The intent of Factory Method is to allow a class to defer instantiation to its subclasses.
 - ▶ The intent of Abstract Factory is to create families of related objects without having to depend on their concrete classes.
- ▶ The Dependency Inversion Principle guides us to avoid dependencies on concrete types and to strive for abstractions.
 - ▶ Factories are a powerful technique for coding to abstractions, not concrete classes.



References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, O'Reilly Media 2004
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.